Painting with Darts: Fine-Tuning $\pi_{0.7}$ for Language-Conditioned Aiming on an SO-101 Arm*

Vineeth Sendilraj Vardhan Agnihotri Yuvraj Lakhotia

Act-athon 2026, Actor Labs × Physical Intelligence (3rd place)

Abstract

We fine-tune the $\pi_{0.7}$ vision-language-action model to aim a motorized dart blaster, held by a low-cost SO-101 arm, at any cell of a 5×5 grid given the instruction "shoot ⟨cell⟩". A language model (Claude) sits above the policy: it chooses cells to draw a requested picture, inspects the board after each shot and re-issues shots that miss. The policy is trained on 116 teleoperated demonstrations, 16 minutes of data collected in a single session. Across 13 fine-tuning runs, validation error is lowest after about 33 passes over the data (0.287), and every probed checkpoint aims in the commanded row and column order when only the instruction changes. The system completed its live drawing demonstration. We release the dataset in LeRobot format under CC BY 4.0, together with the code, wiring and fine-tuning logs.

1 Introduction

Vision-language-action (VLA) models such as $\pi_{0.7}$ [1] can be adapted to a new robot and task with tens of minutes of teleoperated data. A single instruction-following policy does not, however, plan toward a multi-step goal or notice when a step has failed. A common remedy is a hierarchy in which a high-level model chooses the next instruction and a low-level policy executes it [2]. We built such a system during a weekend hackathon around a task whose outcome is easy to check visually but hard to guarantee: hitting specific cells of a foam grid with darts, in order, to draw a shape.

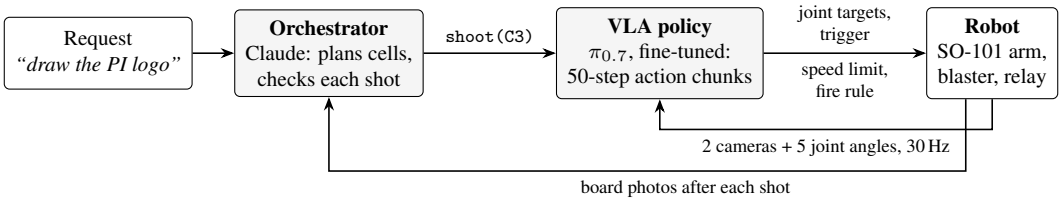


Figure 1. System overview. Claude decides which cell to shoot next and verifies the result from photographs; the fine-tuned $\pi_{0.7}$ policy turns each `shoot(cell)` command and the live camera stream into joint targets and a trigger at 30 Hz.

2 Setup

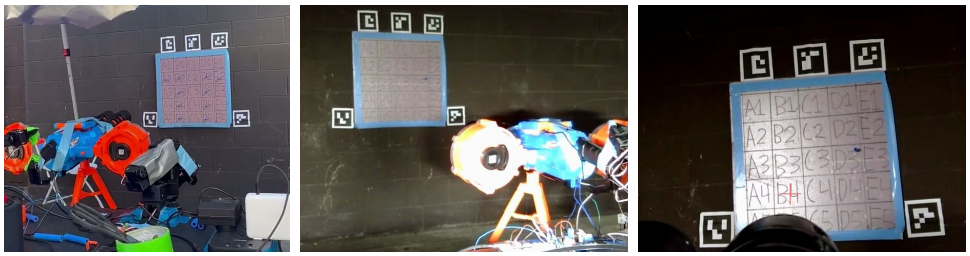


Figure 2. Left: the rig. Middle and right: the policy's rear and blaster-mounted cameras during a `shoot B4` demonstration; the red cross marks the barrel axis.

The follower arm's gripper is replaced by a wire loop around the blaster handle, and an ESP32-driven relay replaces the blaster's trigger switch, so firing is a binary action (Fig. 2); Fig. 1 shows the control loop. The policy observes two 640×480 images (a rear view of the arm and board, and a view from the blaster) and five joint angles at 30 Hz. It outputs chunks of 50 future steps, each with five absolute joint targets and a trigger value. At inference we execute 25 steps per chunk, limit each joint to 3° per step, and fire once the arm is still and the predicted trigger has exceeded 0.5 for three consecutive steps.

*Dataset: `https://huggingface.co/datasets/vineethsendilraj/pi07-dart-painting-so101`. Code, wiring and training logs: `https://github.com/VineethSendilraj/pi07-dart-painting-so101`.

Above the policy, Claude receives the requested picture and photographs from both cameras, and issues one `shoot(cell)` command at a time. After each shot it receives new photographs, so a dart that missed or fell out is detected and the cell is shot again.

3 Dataset

An operator aimed the follower with a leader arm and fired with a physical button. A web application proposed the next cell, board clutter and start pose, and replayed each take frame by frame so that it could be kept or discarded (Fig. 3). The 116 kept demonstrations contain 29,131 frames at 30 Hz (16.2 minutes) and cover 23 of the 25 cells, with A5 and E5 absent. Each frame holds the rear and wrist images, five follower joint angles, and an action of five leader joint angles plus a trigger that is 1 for the 10 steps after the button press. The set mixes 23 live and 93 dry shots, three levels of board clutter and 17 start poses, and is released in LeRobot v3.0 format [3] under CC BY 4.0.

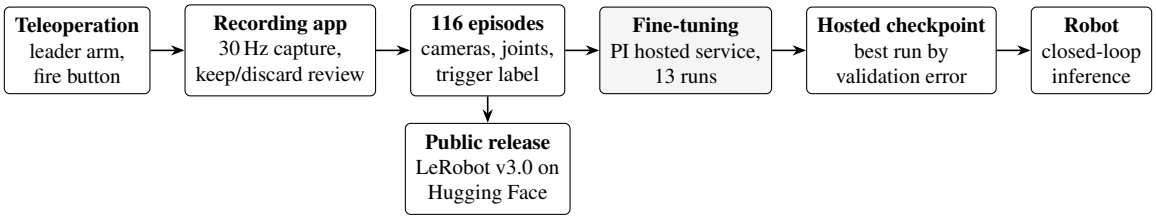


Figure 3. Data and training pipeline. The same episodes are exported to LeRobot format for public release.

4 Training and Results

We ran 13 fine-tuning jobs on Physical Intelligence's hosted service, varying the training schedule, batch size, data mixture and base checkpoint (Table 1). The training schedule mattered most: validation error was lowest after about 33 passes over the data on the standard base checkpoint and about 18 passes on an alternative base, and longer training overfit. A larger batch and oversampling the cells of the PI logo did not help, and training only on the 101 takes recorded with the final crosshair overlay was no better than using all 116. The run with the lowest error on the standard base was used for the live demonstration.

Table 1. Validation error of all 13 fine-tuning runs (best checkpoint per run), by base checkpoint.

Standard base			*Alternative base*		
Change from baseline	Passes	Val. error	Change from baseline	Passes	Val. error
None (baseline)	26.5	0.298	None	26.5	0.294
Longer schedule (demo)	33.5	**0.287**	Shorter schedule	17.6	0.294
Much longer schedule	48.5	0.303	Logo cells oversampled	17.6	0.281[c]
Batch size 512	48.5	0.334	Final-crosshair takes only	∼47	0.294[a]
Logo cells oversampled	33.5	0.333	Still frames dropped	17.6	0.375[b]
Final-crosshair takes only	∼47	0.325[a]	Logo + still frames dropped	17.6	0.444[b]
Still frames dropped	26.5	0.421[b]			

[a] Validated on the 101-take subset. [b] Validated on moving frames only. [c] The validation split likely over-represents the logo cells. Scores marked a–c are not directly comparable with the others.

To test whether the policy uses the instruction, we gave each checkpoint the same images and joint state while changing only the cell in the prompt. All 24 probed checkpoints produced aims ordered by column and row, so the policy follows the instruction rather than replaying a fixed motion.

5 Limitations

We did not measure a hit rate on the real system; our evidence is validation error, the prompt probe and the live demonstration. Because each demonstration continues for 2 s after the shot with the trigger at 0, the policy rarely produced a clean trigger pulse offline; the stillness rule in Section 2 compensates. Coverage is uneven (two demonstrations each for row 1, A3 and E3), and the camera and board positions were fixed throughout.

References

[1] Physical Intelligence. $\pi_{0.7}$: A steerable generalist robotic foundation model with emergent capabilities. arXiv:2604.15483, 2026.

[2] L. X. Shi, B. Ichter, M. Equi, et al. Hi Robot: Open-ended instruction following with hierarchical vision-language-action models. In *ICML*, 2025.

[3] R. Cadene, S. Alibert, A. Soare, et al. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. `https://github.com/huggingface/lerobot`, 2024.